

JPEG编码详解

JPEG 压缩你可以把它理解成一句话：

把人眼不太敏感的信息尽量丢掉，把剩下的信息用更省空间的方式表示。

它不是简单地“压缩文件”，而是一步步把图像变成更容易压缩的数据。典型 JPEG 编码流程如下：

RGB 图像

↓

颜色空间转换 RGB → YCbCr

↓

色度下采样

↓

分块：8×8

↓

DCT 离散余弦变换

↓

量化

↓

Zig-Zag 扫描

↓

游程编码 RLE

↓

熵编码 Huffman / 算术编码

↓

JPEG 压缩数据



1. RGB 转 YCbCr

原图一般是 RGB：

R: 红色

G: 绿色

B: 蓝色

JPEG 通常先把它转换成 YCbCr：

Y : 亮度信息
Cb : 蓝色色度差
Cr : 红色色度差

也就是把图像拆成:

亮不亮 + 颜色偏蓝多少 + 颜色偏红多少

设计用意

人眼对**亮度变化**很敏感, 但对**颜色细节**没那么敏感。

比如一张照片稍微模糊一点颜色, 你可能看不出来; 但是如果亮暗边缘糊了, 你马上觉得“不清楚”。

所以 JPEG 先把亮度和色度分开, 方便后面:

亮度 **Y**: 少丢一点
色度 **Cb/Cr**: 可以多压缩一点

这一步本身通常不是主要压缩来源, 主要是为了给后面的压缩做准备。

2. 色度下采样

既然人眼对颜色细节不敏感, 那 JPEG 会减少 Cb、Cr 的采样数量。

常见格式有:

4:4:4 不下采样
4:2:2 水平方向色度减半
4:2:0 水平和垂直方向色度都减半

比如 4:2:0 的意思大概是:

4 个亮度 **Y** 像素
共用较少的 **Cb、Cr** 色度信息

设计用意

减少色度数据量。

因为颜色细节少一点，人眼通常不明显，所以这是 JPEG 中一个比较“划算”的压缩方式。

可以理解成：

亮度认真存，颜色偷懒存

3. 分成 8×8 小块

JPEG 不会直接对整张图一起处理，而是把图像分成很多个 8×8 的小块。

每一块有 64 个像素值：

$$8 \times 8 = 64$$

后面的 DCT、量化、编码都是以 8×8 块为单位进行。

设计用意

主要有两个：

第一，方便计算。

对整张图做变换太复杂，把图切成小块更容易处理。

第二，图像局部通常比较相似。

比如一小块天空、一小块墙面、一小块皮肤，颜色和亮度往往变化不大。这样经过后续变换后，更容易压缩。

不过这也会带来一个副作用：

压缩率太高时会出现明显的“小方块”，也就是 JPEG 的块效应。

4. DCT 离散余弦变换

这是 JPEG 的核心步骤之一。

DCT 会把 8×8 像素块从“空间域”变到“频率域”。

别被这个词吓到，它的意思是：

原来我们存的是：

每个位置的像素值是多少

经过 DCT 后，变成存：

这一块整体平均亮度是多少
有没有缓慢变化
有没有剧烈变化
有没有边缘、纹理、噪声

DCT 到底怎么算

对 JPEG 来说，DCT 通常是对一个 8×8 图像块计算的。

设这个 8×8 块中第 x 行、第 y 列的像素值为：

$$f(x, y)$$

在 JPEG 里，这里的 $f(x, y)$ 通常不是原始的 $0 \sim 255$ 像素值，而是先做过电平平移后的值：

$$f(x, y) = \text{原像素值} - 128$$

也就是把 $[0, 255]$ 变成 $[-128, 127]$ ，这样更适合后面做余弦变换。

其中：

$$\begin{aligned} x &= 0, 1, 2, \dots, 7 \\ y &= 0, 1, 2, \dots, 7 \end{aligned}$$

经过 DCT 后，会得到另一个 8×8 的系数矩阵：

$$F(u, v)$$

其中：

$$\begin{aligned} u &= 0, 1, 2, \dots, 7 \\ v &= 0, 1, 2, \dots, 7 \end{aligned}$$

二维 DCT 的计算公式是：

$$\begin{aligned} F(u, v) &= 1/4 \cdot C(u) \cdot C(v) \\ &\quad \cdot \sum_{x=0..7} \sum_{y=0..7} f(x, y) \end{aligned}$$

$$\begin{aligned} & \cdot \cos((2x+1)u\pi / 16) \\ & \cdot \cos((2y+1)v\pi / 16) \end{aligned}$$

其中：

$$\begin{aligned} C(0) &= 1/\sqrt{2} \\ C(k) &= 1, k \neq 0 \end{aligned}$$

如果写成更口语的形式，就是：

某个 DCT 系数
= 原图像块的 64 个像素
分别乘上某种余弦波形模板
再全部加起来

所以 DCT 的本质不是“猜”这块图像哪里是低频、哪里是高频，而是用一组固定的余弦函数去和图像块做匹配。

DCT 后得到的 64 个系数可以分成两类：

低频系数：表示整体、平滑变化
高频系数：表示边缘、细节、纹理

其中左上角的系数叫 **DC 系数**，表示这一整块的平均亮度。

其他 63 个叫 **AC 系数**，表示这一块中的变化和细节。

“余弦模板匹配”是什么意思

可以把每一个 (u, v) 看成一种固定的 8×8 余弦模板。

u 控制水平方向变化有多快
v 控制垂直方向变化有多快

比如：

F(0,0)：水平和垂直方向都不变化

它对应的是一块完全平坦的模板，所以它反映整块图像的平均亮度，也就是 DC 系数。

再比如：

$F(1,0)$ 、 $F(2,0)$ ：主要表示水平方向上较慢或较快的亮暗变化

$F(0,1)$ 、 $F(0,2)$ ：主要表示垂直方向上较慢或较快的亮暗变化

$F(7,7)$ ：水平和垂直方向都变化很快

如果某个图像块和某个余弦模板很像，那么对应的 DCT 系数就会比较大。

如果不像，正负部分会互相抵消，最后加出来的系数就会比较小。

这就是：

DCT 不是凭感觉判断高频，
而是用不同频率的余弦模板去匹配图像。

更直观地说：

平滑区域：更像低频模板 → 低频系数大，高频系数小

边缘纹理：更像高频模板 → 高频系数可能变大

JPEG 图像中很多 8×8 小块都比较平滑，所以 DCT 后往往会出现：

左上角低频系数比较大

右下角高频系数比较小

这就为后面的量化创造了条件：高频系数小，而且人眼不太敏感，所以更容易被压成 0。

设计用意

DCT 的目的不是直接压缩，而是为了让数据变得“好压缩”。

普通像素值可能看起来乱乱的，但 DCT 后会出现一个特点：

重要信息集中在低频

不太重要的信息集中在高频

而人眼对高频细节，尤其是颜色高频细节，不太敏感。

所以后面就可以对高频信息狠一点压缩。

一句话：

DCT 负责把“哪里能丢”暴露出来。

5. 量化

量化是 JPEG 有损压缩的关键步骤。

DCT 后得到的是一堆系数，JPEG 会用量化表去除它们：

量化后系数 = DCT 系数 $\div$ 量化表中的数，再四舍五入

例如：

DCT 系数 = 83

量化值 = 10

$83 \div 10 \approx 8$

如果某个高频系数本来就不大，再除以一个较大的量化值，就可能变成 0。

比如：

DCT 系数 = 6

量化值 = 30

$6 \div 30 \approx 0$

这样很多高频细节会直接变成 0。

设计用意

这一招很狠，但很有效。

量化表通常会设计成：

低频部分：量化值小，保留得多

高频部分：量化值大，丢得多

因为低频决定图像整体观感，高频多是细节、纹理、噪声。

所以量化的本质是：

保留重要的低频信息

削弱甚至丢弃不重要的高频信息

JPEG 的“质量因子”主要就是控制量化强度。

质量高：

量化弱 → 丢得少 → 文件大 → 图像清晰

质量低：

量化强 → 丢得多 → 文件小 → 图像失真明显

6. Zig-Zag 扫描

量化之后，每个 8×8 块还是 64 个数字。

JPEG 会按照 Zig-Zag 顺序把二维矩阵变成一维序列。

顺序大概是从左上角开始，沿着斜线走：

先低频 → 再中频 → 最后高频

因为 DCT 系数矩阵中：

左上角：低频，通常非零多

右下角：高频，量化后很多是 0

设计用意

Zig-Zag 扫描是为了把很多 0 聚集在序列后面。

例如扫描后可能得到：

15, -2, 1, 0, 3, 0, 0, 0, 0, 0, 0 ...

越往后越多 0。

这样下一步游程编码就很舒服。

一句话：

Zig-Zag 是为了制造连续的 0。

7. 游程编码 RLE

游程编码主要用来压缩连续出现的 0。

比如原序列是：

```
5, 0, 0, 0, 0, 2
```

可以表示为：

```
5, 后面有 4 个 0, 然后是 2
```

JPEG 对 AC 系数尤其适合用这种方式，因为量化后高频系数很多都会变成 0。

设计用意

减少连续 0 的存储空间。

经过 Zig-Zag 后，0 往往集中在末尾，所以游程编码效果很好。

如果一块后面全是 0，还可以用一个特殊符号：

```
EOB: End Of Block, 块结束
```

意思是：

```
后面全是 0, 不用再写了
```

这个设计很省空间。

8. DC 系数差分编码

每个 8×8 块左上角的 DC 系数表示该块的平均亮度。

相邻图像块的平均亮度通常差不多，所以 JPEG 不直接存 DC 系数，而是存相邻块 DC 系数的差值。

比如：

```
第 1 块 DC = 120
```

```
第 2 块 DC = 123
```

```
第 3 块 DC = 125
```

不存：

120, 123, 125

而是存：

120, 3, 2

设计用意

差值通常比原值小。

小数更容易用更短的编码表示，所以可以进一步压缩。

9. 熵编码：Huffman 编码 / 算术编码

最后一步是无损压缩。

前面的量化已经造成信息损失了，但熵编码本身不再丢信息。

JPEG 常用 Huffman 编码：

出现频率高的符号 → 用短码

出现频率低的符号 → 用长码

比如大量出现的 0、EOB、小差值，就可以用很短的码表示。

设计用意

进一步减少比特数。

这一阶段相当于：

不改变数据含义，只是换一种更省空间的写法

所以 JPEG 中：

有损部分：色度下采样、量化

无损部分：RLE、差分编码、Huffman 编码

其中最主要的有损来源是：

量化

总结版：每一步干什么

步骤	做了什么	设计用意
RGB → YCbCr	分离亮度和色度	利用人眼对亮度敏感、对色度不敏感
色度下采样	减少 Cb/Cr 数据	丢掉不明显的颜色细节
8×8 分块	把图像分块处理	降低计算复杂度，利用局部相似性
DCT	空间域变频率域	把重要信息集中到低频
量化	DCT 系数除以量化表并取整	丢掉不重要的高频信息，是有损核心
Zig-Zag	按低频到高频顺序扫描	让 0 尽量连续出现
RLE	压缩连续的 0	减少冗余
DC 差分编码	存相邻块 DC 差值	利用相邻块平均亮度相近
Huffman 编码	高频符号短编码	无损地进一步压缩

考试背诵版

可以这样背：

JPEG 先将 RGB 转换为 YCbCr，并对色度分量进行下采样，利用人眼对亮度敏感而对色度不敏感的特点减少数据量。然后将图像划分为 8×8 小块，对每个块进行 DCT 变换，把空间域像素转换为频率域系数，使能量主要集中在低频部分。接着进行量化，对低频系数保留较多，对高频系数压缩较强，这是 JPEG 有损压缩的主要来源。量化后的系数经过 Zig-Zag 扫描，使大量 0 集中出现，再对 AC 系数进行游程编码，对 DC 系数进行差分编码，最后使用 Huffman 等熵编码实现进一步无损压缩。

最关键的一句：

DCT 负责把图像信息分成低频和高频；
 量化负责真正丢掉不重要的高频信息；
 后面的 Zig-Zag、RLE、Huffman 负责把剩下的数据无损压得更小。